Fiche de cours JavaScript : variables (var, let, const) et types

Lycée Rafèha, 3ème et 4ème SI. Enseignant : Aymen Essouyah

1. Déclarer une variable

Une variable est un espace mémoire nommé. En JavaScript, trois mots-clés permettent de la déclarer.

Mot-clé	Portée	Redéclaration	Réaffectation	Avant la déclaration
`var`	Fonction (ou globale)	Autorisée	Oui	`undefined`
`let`	Bloc `{ }`	Interdite (même bloc)	Oui	`ReferenceError` (TDZ)
`const`	Bloc `{ }`	Interdite	**Non**	`ReferenceError` (TDZ)

Règle pratique : `const` par défaut, `let` si la valeur doit changer, éviter `var`.

2. La portée (scope)

Portée (en anglais *scope*) : c'est la zone du programme où une variable existe et où l'on peut l'utiliser. En dehors de cette zone, JavaScript ne la connaît pas.

Analogie du lycée : la portée, c'est la distance que la voix peut atteindre.

- Une annonce au haut-parleur est entendue dans toutes les salles : variable **globale**.
- Ce que l'enseignant dit dans la salle 12 n'est entendu que dans la salle 12 : variable **locale** (dans une fonction).
- Ce que dit un groupe de travail à l'intérieur de la salle n'est entendu que par ce groupe : variable **de bloc** (`let` et `const`). Avec `var`, il n'y a pas de groupes : tout ce qui est dit est entendu dans toute la salle.

Un élève de la salle 12 entend l'annonce générale, mais le reste du lycée n'entend pas ce qui se dit dans la salle 12.

Globale : variable déclarée hors de toute fonction, accessible partout.

```
let site = "STI";
function afficher() {
  console.log(site); // accessible
}
```

Locale : variable déclarée dans une fonction, inaccessible à l'extérieur.

```
function test() {
  var x = 10;
}
console.log(x); // ReferenceError
```

De bloc (`let` et `const`) : limitée aux accolades `{ }` d'un `if`, d'une boucle, etc.

```
if (true) {
  let a = 1;
  var b = 2;
}
console.log(b); // 2 (var ignore le bloc)
console.log(a); // ReferenceError
```
Une fonction accède aux variables de ses fonctions parentes, jamais l'inverse (chaîne de portée).

3. Le mot-clé `let`

```
let age = 16;
age = 17; // autorisé

let x = 1;
let x = 2; // SyntaxError (même bloc)

let note;
console.log(note); // undefined
```
Zone morte temporelle (TDZ) : entre le début du bloc et la ligne de déclaration, la variable existe mais n'est pas utilisable.

```
console.log(n); // ReferenceError
let n = 5;
```
Dans une boucle, chaque itération a sa propre copie :

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
for (let j = 0; j < 3; j++) setTimeout(() => console.log(j)); // 0 1 2
```

4. Le mot-clé `const`

```
const PI = 3.14;
PI = 3.15;   // TypeError
const y;     // SyntaxError (initialisation obligatoire)
```
La liaison est constante, pas le contenu : on ne peut pas réaffecter la variable, mais on peut modifier le contenu d'un tableau (une case t[i]).

```
const notes = [12, 15, 9];
notes[0] = 14;          // autorisé (on modifie une case)
notes = [10, 11];       // TypeError (réaffectation)
```
`const` est possible dans `for...of`, mais pas dans un `for` à compteur :

```
for (const note of [12, 15, 9]) console.log(note); // correct
for (const i = 0; i < 3; i++) {}                   // TypeError
```

5. Le hoisting (remontée)

Hoisting (remontée) : avant d'exécuter le code, JavaScript parcourt tout le programme et **note les déclarations** de variables, comme si elles étaient « remontées » en haut de leur zone. Seule la **déclaration** remonte, pas la valeur donnée ensuite.

Analogie de la classe : avant le début du cours, le surveillant lit la liste des élèves inscrits. Tous les noms sont connus dès le départ, même si les élèves ne sont pas encore arrivés.

- **`var`** : l'élève est déjà assis à sa place, mais son sac est vide. Si on l'appelle, il répond « `undefined` ».
- **`let` et `const`** : l'élève est inscrit, mais il attend encore dans le couloir (zone morte temporelle). L'appeler avant son arrivée provoque une erreur (`ReferenceError`). Il devient utilisable à la ligne où la variable est déclarée : c'est son entrée en classe.

```
console.log(x); // undefined
var x = 5;

console.log(y); // ReferenceError (TDZ)
let y = 5;
```

Avec `var`, la déclaration est remontée et initialisée à `undefined`. Avec `let` et `const`, elle est remontée mais inutilisable jusqu'à la ligne de déclaration.

6. Les types de données

JavaScript est à **typage dynamique** : le type est attaché à la **valeur**, pas à la variable.

```
let x = 10;   // number
x = "dix";    // string (autorisé)
```

Types primitifs (7)

Primitif vient du latin *primitivus* : « premier », « de base ». Un type primitif est une valeur **simple et indivisible**, qui ne contient rien d'autre.

Analogie de la cuisine : le sel, la farine et l'œuf sont des ingrédients de base, on ne peut pas les décomposer en quelque chose de plus simple. Un gâteau, lui, est composé de plusieurs ingrédients : c'est un **objet** (tableau, fonction, date). Deux sacs de farine identiques sont indépendants : vider l'un ne vide pas l'autre, comme pour la copie d'un primitif.

Type	Exemple	Remarque
`string`	`"Ali"`, `` `Bonjour ${nom}` ``	Texte
`number`	`15`, `3.14`, `NaN`, `Infinity`	Entiers et réels (un seul type)
`boolean`	`true`, `false`	
`undefined`	`let a;`	Variable sans valeur
`null`	`let u = null;`	Absence volontaire de valeur
`bigint`	`123456789012345678901234567890n`	Très grands entiers
`symbol`	`Symbol("id")`	Identifiant unique (usage avancé)

Types objets

`Object`, `Array`, `Function`, `Date`, `RegExp`, `Map`, `Set`...

```
const notes = [12, 15, 9];   // Array
const carre = (x) => x * x;  // Function
const d = new Date();        // Date
```

L'opérateur `typeof`

```
typeof "texte";      // "string"
typeof 42;           // "number"
typeof true;         // "boolean"
typeof undefined;    // "undefined"
typeof 10n;          // "bigint"
typeof Symbol();     // "symbol"
typeof [];           // "object"   (piège : utiliser Array.isArray)
typeof (() => 1);    // "function"
typeof null;         // "object"   (bug historique : tester x === null)
```

Valeur ou référence

Les primitifs sont copiés **par valeur**, les objets **par référence**.

```
let a = 5;
let b = a;
b = 10;
console.log(a); // 5

const t1 = [1, 2, 3];
const t2 = t1;
t2[0] = 99;
console.log(t1[0]); // 99 (même tableau)

// copie indépendante : recopier case par case
const copie = [];
for (let i = 0; i < t1.length; i++) {
  copie[i] = t1[i];
}
copie[0] = 0;
console.log(t1[0]); // 99 (t1 n'a pas changé)
```

7. Conversions de types

Explicites

```
Number("42");      // 42
Number("abc");     // NaN
parseInt("15px");  // 15
parseFloat("3.5"); // 3.5
String(123);       // "123"
Boolean(0);        // false
```

Implicites (coercition)

```
"5" + 2;   // "52" (concaténation)
"5" - 2;   // 3    (conversion en nombre)
true + 1;  // 2
```

8. Valeurs falsy et truthy (hors programme, mais à connaître)

Falsy : `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. Tout le reste est **truthy**, y compris `"0"`, `"false"` et `[]`.

9. Comparaison : `==` et `===`

```
5 == "5";            // true  (avec coercition)
5 === "5";           // false (type différent)
null == undefined;   // true
null === undefined;  // false
NaN === NaN;         // false
Number.isNaN(NaN);   // true
```

Règle : utiliser `===` et `!==`.

10. À retenir

- `const` par défaut, `let` si réaffectation, `var` à éviter.
- `let` et `const` ont une portée de bloc et une zone morte temporelle.
- `const` empêche la réaffectation, pas la modification du contenu d'un tableau.
- 7 types primitifs, tout le reste est un objet.
- `typeof null` vaut `"object"` et `typeof []` aussi : cas particuliers à connaître.
- Préférer `===` à `==`.

Exercices

Exercice 1 : Que s'affiche-t-il ?

Pour chaque extrait, donner ce qui s'affiche dans la console (ou l'erreur produite).

```
// a)
var a = 1;
{
  var a = 2;
}
console.log(a);

// b)
let b = 1;
{
  let b = 2;
}
console.log(b);

// c)
console.log(c);
var c = 5;

// d)
console.log(d);
let d = 5;
```

```
// e)
const e = 10;
e = 20;
console.log(e);

// f)
const f = [1, 2];
f[0] = 5;
console.log(f);
```

Exercice 2 : Corriger les erreurs

```
const total;
total = 0;
for (const i = 0; i < 3; i++) {
  total += i;
}
let total = 5;
```

Exercice 3 : typeof

Donner le résultat de chaque expression.

```
typeof 3.14;
typeof "3.14";
typeof null;
typeof [1, 2];
typeof undefined;
typeof NaN;
typeof (() => 1);
```

Exercice 4 : Conversions et comparaisons

```
// a)
console.log("10" + 5);
// b)
console.log("10" - 5);
// c)
console.log(5 == "5");
// d)
console.log(5 === "5");
// e)
console.log(Number("abc"));
```

~~f) console.log(Boolean("0"));~~ (hors programme)

```
// g)
console.log(0.1 + 0.2 === 0.3);
```

Exercice 5 : Boucle et portée

Que s'affiche-t-il avec chaque version ? Expliquer la différence.

```
// Version 1
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

```
// Version 2
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 0);
}
```

Exercice 6 : Programmation

Écrire une fonction `moyenne(notes)` qui reçoit un tableau de nombres et retourne leur moyenne. Contraintes : utiliser `const` ou `let` selon le cas, une boucle `for`, et ne déclarer aucune variable globale à l'intérieur de la fonction. Tester avec `[12, 15, 9]`.

Exercice 7 : Valeur ou référence

```
let x = 5;
let y = x;
y = 8;

const p = [15, 16];
const q = p;
q[0] = 18;

console.log(x, y);
console.log(p[0], q[0]);
```

Que s'affiche-t-il ? Comment obtenir une copie indépendante de `p` ?

Corrigés

Corrigé de l'exercice 1

	Résultat	Explication
a)	`2`	`var` ignore le bloc et redéclare la même variable
b)	`1`	Le `let b` du bloc est une autre variable
c)	`undefined`	Hoisting de `var` : déclarée, pas encore initialisée
d)	`ReferenceError`	`let` dans la zone morte temporelle
e)	`TypeError`	Réaffectation d'une constante
f)	`[5, 2]`	`f[0] = 5` modifie le contenu, pas la liaison

Corrigé de l'exercice 2

Erreurs : `const total;` sans initialisation ; réaffectation de `total` ; `const i` incrémenté dans un `for` ; redéclaration de `total` avec `let`.

```
let total = 0;
for (let i = 0; i < 3; i++) {
  total += i;
```

```
}
console.log(total); // 3
```

Corrigé de l'exercice 3

Expression	Résultat
`typeof 3.14`	`"number"`
`typeof "3.14"`	`"string"`
`typeof null`	`"object"`
`typeof [1, 2]`	`"object"`
`typeof undefined`	`"undefined"`
`typeof NaN`	`"number"`
`typeof (() => 1)`	`"function"`

Corrigé de l'exercice 4

	Résultat	Explication
a)	`"105"`	`+` avec une chaîne : concaténation
b)	`5`	`-` convertit en nombre
c)	`true`	`==` fait une coercition
d)	`false`	Types différents
e)	`NaN`	Conversion impossible
~~f)~~	~~`true`~~	~~`"0"` est une chaîne non vide, donc truthy~~ (hors programme)
g)	`false`	`0.1 + 0.2` vaut `0.30000000000000004`

Corrigé de l'exercice 5

- Version 1 : `3 3 3`. Avec `var`, il n'y a qu'une seule variable `i` pour toute la boucle ; quand les fonctions s'exécutent, la boucle est terminée et `i` vaut 3.
- Version 2 : `0 1 2`. Avec `let`, chaque itération crée sa propre variable `j`, conservée par la fonction (closure).

Corrigé de l'exercice 6

```
function moyenne(notes) {
  let somme = 0;
  for (let i = 0; i < notes.length; i++) {
    somme += notes[i];
  }
  return somme / notes.length;
}
```

```
console.log(moyenne([12, 15, 9])); // 12
```
`somme` est un `let` car sa valeur change ; `i` et `somme` sont locaux à la fonction.

Corrigé de l'exercice 7

```
5 8
18 18
```

- `x` et `y` sont des primitifs : copie par valeur, `x` reste à 5.
- `p` et `q` désignent le même tableau : copie par référence, donc `p[0]` vaut aussi 18.
- Copie indépendante : recopier case par case avec une boucle.

```
const copie = [];
for (let i = 0; i < p.length; i++) {
  copie[i] = p[i];
}
```